

Proje Ana Alanı : BİLİŞİM

Proje Tematik Alanı : YAPAY ZEKA VE MAKİNE ÖĞRENMESİ

Proje Adı (Başlığı) : Yapay Zekâ Eğitiminde Gradyan İnişi Algoritmasının Uygulamalı Analizi

Özet:

Bu projede, yapay zekâ ve makine öğrenmesi modellerinin öğrenme sürecinde önemli bir role sahip olan gradyan inişi algoritmasının çalışma prensibi ve eğitim sürecindeki etkisi incelenmiştir. Çalışmanın ilk aşamasında, gradyan inişi algoritmasının matematiksel temeli ele alınmış ve hata fonksiyonunun türevi kullanılarak model parametrelerinin nasıl güncellendiği açıklanmıştır.

Deneysel aşamada, gradyan inişi algoritması Python programlama dili kullanılarak uygulanmıştır. Öncelikle algoritmanın temel mantığını göstermek amacıyla basit bir matematiksel fonksiyon üzerinde denemeler yapılmış, ardından küçük bir veri seti ile eğitilen bir makine öğrenmesi modeli üzerinde test edilmiştir. Bu süreçte, toplu, stokastik ve mini-toplu olmak üzere üç farklı gradyan inişi türü kullanılarak modelin eğitim sürecindeki hata değerleri incelenmiş ve elde edilen sonuçlar grafikler yardımıyla görselleştirilmiştir. Ayrıca, farklı öğrenme oranlarının modelin öğrenme süreci üzerindeki etkisi karşılaştırılmıştır.

Elde edilen bulgular, gradyan inişi algoritmasının her iterasyonda hata fonksiyonunu azaltarak model parametrelerini daha uygun değerlere yönlendirdiğini göstermiştir. Farklı gradyan inişi türlerinin benzer bir hata azalımı sağladığı, ancak öğrenme sürecinin hızı ve yakınsamanın öğrenme oranına bağlı olarak değiştiği gözlemlenmiştir.

Sonuç olarak, bu çalışma gradyan inişi algoritmasının yapay zekâ modellerinin öğrenme sürecindeki temel rolünü hem matematiksel hem de deneysel açıdan ortaya koymaktadır.

Anahtar Kelimeler: Makine öğrenmesi, Gradyan inişi, Optimizasyon, Hata fonksiyonu, Öğrenme oranı

Amaç:

Bu projenin amacı, yapay zekâ ve makine öğrenimi modellerinin eğitimi sürecinde kullanılan gradyan inişi algoritmasının makine öğrenme üzerindeki etkisini incelemek, algoritmanın hata azaltma göstermek ve farklı gradyan inişi türlerinin performans farklılıklarını karşılaştırmaktır.

Bu kapsamda, seçilen makine öğrenimi modelinin hata fonksiyonunu nasıl azalttığı, parametreleri nasıl güncellediği ve öğrenme oranı gibi ayarların öğrenme sürecini hangi yönde etkilediği düzenli bir biçimde incelenmiştir. Bu amaçla üç farklı optimizasyon yaklaşımının—toplu gradyan inişi, rastgele gradyan inişi ve küçük gruplu gradyan inişi—aynı problem üzerinde uygulanması ve performanslarının karşılaştırılması hedeflenmektedir.

Bu çalışma, yapay zekâ algoritmalarının “nasıl öğrendiği” konusunda literatürde sıkça kullanılan ancak lise düzeyinde sınırlı sayıda uygulamalı çalışma bulunan gradyan inişi yöntemini hem matematiksel hem de deneysel açıdan ortaya koyarak önemli bir eksikliği gidermeyi amaçlamaktadır. Proje sonucunda, farklı optimizasyon stratejilerinin hata azaltma

davranışlarının açık şekilde gözlemlenmesi, yapay zekâ modellerinin eğitim süreçlerinin daha iyi anlaşılmasına katkı sağlayacaktır.

Giriş:

Yapay zeka ve aynı zamanda makine öğrenmesinin temel algoritması veriler üzerinden öğrenme yaparak tahmin ve sınıflandırma görevlerini yerine getirmektir. Bu algorithmada doğru sonuçlar, öğrenme sürecinde hata fonksiyonunun azaltılmasına bağlıdır. Hata fonksiyonu ise bu konuda kullanılan en yaygın yöntemlerden biri olan gradyan inişi yöntemi kullanılarak azaltılır (Bishop, 2006). Gradyan inişi, parametrelerini hata fonksiyonunun gradyanına göre güncelleyerek modelin daha düşük hata değerlerine doğru bir şekilde ilerlemesini sağlar.

Araştırmalar, gradyan inişinin doğrusal ve doğrusal olmayan modellerde yüksek etkinlik gösterdiğini, ancak seçilen hiperparametrelerin, özellikle öğrenme oranının, yakınsama sürecini önemli ölçüde etkilediğini ortaya koymaktadır (Goodfellow, Bengio, & Courville, 2016). Gradyan inişinin üç temel türü—toplu, stokastik ve mini-toplu gruplu güncellemeye dayalı yaklaşımlar—eğitim hızı, stabilite ve genel performans açısından birbirinden farklı sonuçlar üretebilmektedir (Ruder, 2016). Buna karşın, bu yöntemlerin farklı veri işleme stratejileri altında nasıl davrandığını lise düzeyinde açık ve uygulamalı şekilde ortaya koyan çalışmalar sınırlıdır.

Bu proje, gradyan inişi algoritmasının yapay zekâ modellerinin öğrenme sürecindeki rolünü hem matematiksel hem de deneysel açıdan incelemeyi ve farklı gradyan inişi türlerinin öğrenme performansı üzerindeki etkilerini uygulamalı örnekler üzerinden analiz etmeyi amaçlamaktadır.

Yöntem:

Gradyan İnişi Algoritmasının Matematiksel Temeli

Gradyan inişi algoritması, bir hata fonksiyonunu minimize etmeye yönelik yinelemeli bir optimizasyon yöntemidir. Bu algorithmada amaç, modelin tahminleri ile gerçek değerler arasındaki farkı ifade eden hata fonksiyonunun en küçük değerini bulmaktır. Genel olarak minimize edilmek istenen hata fonksiyonu $J(\theta)$ ile gösterilir ve θ , modelin öğrenilen parametrelerini temsil eder.

Gradyan inişi yönteminde parametreler, hata fonksiyonunun gradyanı kullanılarak aşağıdaki güncelleme kuralına göre adım adım güncellenir:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

Bu ifadede η öğrenme oranını, $\nabla J(\theta_t)$ ise hata fonksiyonunun parametrelere göre türevini göstermektedir. Güncelleme işlemi, hata fonksiyonunun en hızlı azaldığı yönde ilerlemeyi amaçlamaktadır.

Bu çalışmada kullanılan model doğrusal regresyon modeli olup aşağıdaki şekilde tanımlanmıştır:

$$y = wx + b$$

Modelin performansını değerlendirmek amacıyla hata fonksiyonu olarak ortalama karesel hata (Mean Squared Error – MSE) kullanılmıştır:

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (y_i - (wx_i + b))^2$$

Gradyan inişi algoritması, bu hata fonksiyonunun w ve b parametrelerine göre hesaplanan gradyanları kullanılarak uygulanmıştır. Böylece model parametreleri, her yinelemede hata değerini azaltacak şekilde güncellenmiştir.

Bu matematiksel çerçeve doğrultusunda, çalışmada Python programlama dili kullanılarak gradyan inişi algoritmasının yapay zekâ modellerinin öğrenme süreci üzerindeki etkisi deneysel olarak incelenmiştir (ekler kısmında yazılan kodun tamamı bulunmaktadır). Bu amaçla, farklı gradyan inişi türleri ve farklı öğrenme oranları kullanılarak model performansı test edilmiştir. Çalışma boyunca tüm parametreler sabit tutulmuş, yalnızca incelenen değişkenler kontrollü olarak değiştirilmiştir.

Veri Setinin Oluşturulması

Veri setinin her defasında aynı sonucu üretmesi için sabit bir rastgelelik tohumu belirlenmiştir. Bağımsız değişkenler belirli aralıklarla rastgele bir şekilde üretilmiştir. Bağımlı değişkenler ise doğrusal bir fonksiyon ve rastgele gürültü kullanılarak hesaplanmıştır.

```
np.random.seed(42)

w_true = 3.0
b_true = 2.0

X = np.random.rand(100, 1) * 10
noise = np.random.randn(100, 1)
y = w_true * X + b_true + noise
```

Figür 1: Veri Setinin Oluşturulduğu Kod Bloğu

Figür 1’de, deneylerde kullanılacak giriş (X) ve çıkış (y) verilerini oluşturmaktadır. Gürültü eklenmesi ise modelin gerçek dünya verilerine benzer bir şekilde öğrenmesini sağlamak amacıyla tercih edilmiştir.

Modelin Tanımlanması ve Hata Fonksiyonu

Deneylerde doğrusal regresyon modeli kullanılmıştır. Modelin öğrenme sürecini değerlendirebilmek için hata fonksiyonu ortalama kare hata (Mean Squared Error – MSE) olarak tercih edilmiştir. Model parametreleri başlangıçta sıfır olarak atanmış ve tüm deneylerde aynı başlangıç değerleri kullanılmıştır.

```
def compute_loss(w, b, X, y):
    y_pred = w * X + b
    return np.mean((y_pred - y) ** 2)
```

Figür 2: Modelin Tanımlanması ve Hata Fonksiyonu Kod Bloğu

Figür 2’deki fonksiyonda model tarafından üretilen tahminler ile gerçek değerler arasındaki fark hesaplanarak her epoch sonunda model performansının izlenmesini sağlamaktadır.

Gradyan İnişi Algoritması

Tüm gradyan inişi algoritmaları aynı başlangıç parametreleri, veri seti ve epoch sayısı ile çalıştırılmıştır. Tüm algoritmalar 100 epoch boyunca çalıştırılmış ve bu değer, modelin yakınsama davranışının net şekilde gözlemlenebilmesi amacıyla seçilmiştir.

Toplu Gradyan İnişi

```
def batch_gradient_descent(X, y, lr=0.001, epochs=100):
    w, b = 0.0, 0.0
    losses = []

    for epoch in range(epochs):
        y_pred = w * X + b
        dw = np.mean((y_pred - y) * X)
        db = np.mean(y_pred - y)

        w -= lr * dw
        b -= lr * db

        loss = compute_loss(w, b, X, y)
        losses.append(loss)

    return w, b, losses
```

Figür 3: Toplu Gradyan İnişi Kod Bloğu

Figür 3’deki algoritmada, parametre güncellemeleri her epoch’ta tüm veri seti kullanılarak gerçekleştirilmiştir. Her epoch sonunda hata değeri hesaplanarak kaydedilmiştir.

Stokastik Gradyan İnişi

```
def stochastic_gradient_descent(X, y, lr=0.001, epochs=100):
    w, b = 0.0, 0.0
    losses = []

    for epoch in range(epochs):
        for i in range(len(X)):
            y_pred = w * X[i] + b
            dw = (y_pred - y[i]) * X[i]
            db = y_pred - y[i]

            w -= lr * dw
            b -= lr * db

        loss = compute_loss(w, b, X, y)
        losses.append(loss)

    return w, b, losses
```

Figür 4: Stokastik Gradyan İnişi Kod Bloğu

Figür 4’deki model parametreleri her bir veri noktası üzerinden güncellenmiştir. Böylece her epoch içerisinde çok sayıda güncelleme yapılmıştır.

Mini-Toplu Gradyan İniş

Figür 5’te veri seti küçük gruplara ayrılarak parametre güncellemeleri gerçekleştirilmiştir.

```
def mini_batch_gradient_descent(X, y, lr=0.001, epochs=100, batch_size=16):
    w, b = 0.0, 0.0
    losses = []

    for epoch in range(epochs):
        indices = np.random.permutation(len(X))
        X_shuffled = X[indices]
        y_shuffled = y[indices]

        for i in range(0, len(X), batch_size):
            X_batch = X_shuffled[i:i+batch_size]
            y_batch = y_shuffled[i:i+batch_size]

            y_pred = w * X_batch + b
            dw = np.mean((y_pred - y_batch) * X_batch)
            db = np.mean(y_pred - y_batch)

            w -= lr * dw
            b -= lr * db

        loss = compute_loss(w, b, X, y)
        losses.append(loss)

    return w, b, losses
```

Figür 5: Mini-Toplu Gradyan İnişi Kod Bloğu

Öğrenme Oranı Deney Sistemi

Gradyan inişi sisteminin öğrenme oranındaki değişikliğe olan etkisini incelemek amacıyla öğrenme oranı parametresi farklı değerler alacak şekilde belirlenmiştir.

```
learning_rates = [0.0001, 0.001, 0.01]
losses_lr = {}
```

Figür 6: Öğrenme Oranları Kod Bloğu

Figür 6'daki her öğrenme oranı için toplu gradyan inişi metodu kullanılarak deneyler gerçekleştirilmiştir. Toplu gradyan inişi metodunun kullanılmasının sebebi, her adımda veri setinin tamamını kullandığı için hata oranındaki değişimleri daha net bir şekilde göstermesidir.

Verilerin Analiz Süreci

```
plt.figure(figsize=(8,5))

for lr, losses in losses_lr.items():
    plt.plot(losses, label=f"Learning Rate = {lr}")

plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.title("Learning Rate'in Gradient Descent Performansına Etkisi")
plt.legend()
plt.show()
```

Figür 7: Verilerin Analiz Edildiği Kod Bloğu

Figür 7'de yaptığımız deneyler sonucunda elde edilen farklı gradyan inişi yöntemlerine ait hata değerleri, epoch sayısına bağlı olarak aynı grafik üzerinde gösterilerek karşılaştırılmıştır.

Deneyler Python programlama dili kullanılarak NumPy ve Matplotlib kütüphaneleri ile gerçekleştirilmiştir.

Proje İş-Zaman Çizelgesi:

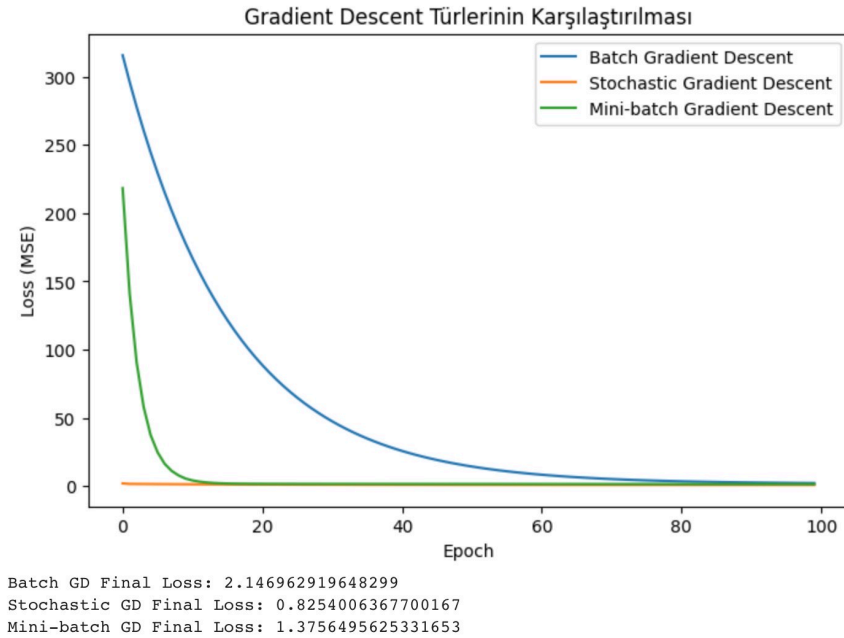
İşin Adı	AYLAR								
	NİSAN	MAYIS	HAZİRAN	TEMMUZ	AĞUSTOS	EYLÜL	EKİM	KASIM	ARALIK
Literatür Taraması	X	X	X	X					
Verilerin Toplanması ve Analizi			X	X	X	X	X		
Proje Raporu Yazımı							X	X	X

Bulgular:

Gradyan İnişi Türlerinin Karşılaştırılmasına Ait Bulgular

Toplu gradyan iniş, stokastik gradyan iniş ve mini toplu gradyan iniş yöntemleri kullanılarak elde edilen hata değerlerinin epoch sayısına bağlı değişimi grafiksel olarak gösterilmiştir.

Şekil 1. Toplu(Batch), stokastik(stochastic) ve mini-toplu(mini-batch) gradyan inişi (gradient descent) yöntemlerine ait hata (MSE) değerlerinin epoch sayısına göre değişimi.



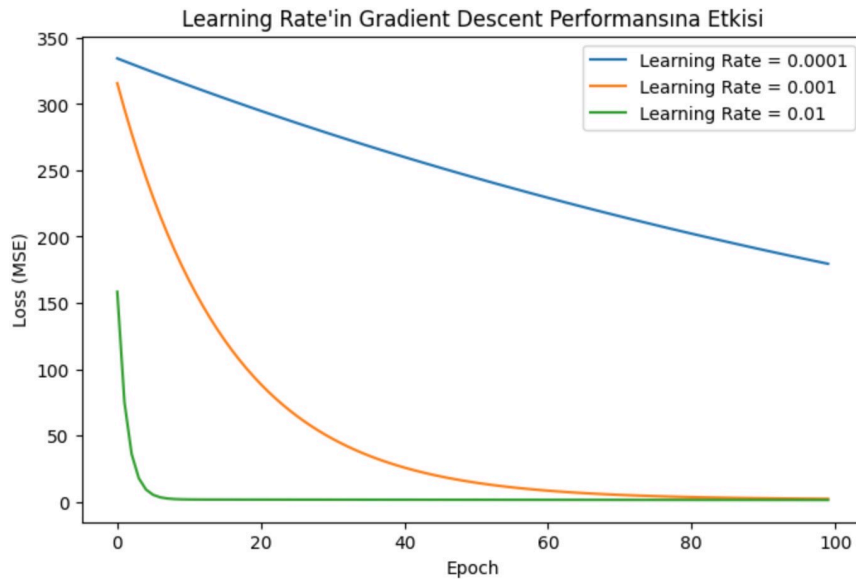
Şekil 1 incelendiğinde, toplu(batch) gradyan iniş yönteminde hata değerlerinin epoch sayısı arttıkça daha düzenli bir biçimde azaldığı görülmektedir. Stokastik(stochastic) gradyan iniş yönteminde hata (MSE) değerlerinin tüm epoch'lar boyunca sıfır seviyesinde sabit kaldığı ve belirgin bir değişim göstermediği görülmektedir. Mini-toplu(mini-batch) gradyan iniş yönteminde ise hata değerlerinin değişimi, toplu ve stokastik yöntemler arasında bir davranış sergilemiştir. Bu sonuçlar, farklı gradyan iniş türlerinin hata değerlerinin değişim biçimleri

açısından birbirinden ayrıldığını göstermektedir. Elde edilen sonuçlar farklı gradyan yöntemlerinin hata değerlerinin yakınsama süreci ve kararlılığı açısından birbirinden ayrıldığını göstermektedir. Bu durum, gradyan inişi algoritmasının uygulanma biçiminin öğrenme sürecinin genel yapısını etkilediğini ortaya koymaktadır.

Öğrenme Oranının Gradyan İnişi Performansına Etkisi

Öğrenme oranının gradyan inişi algoritmasının performansı üzerindeki etkisini incelemek amacıyla, toplu gradyan inişi yöntemi farklı öğrenme oranları kullanılarak uygulanmıştır. Deneyler sonucunda elde edilen hata (MSE) değerlerinin epoch sayısına bağlı değişimi grafiksel olarak sunulmuştur.

Şekil 2. Farklı öğrenme oranları(learning rate) altında toplu(batch) gradyan inişi yöntemine ait hata (MSE) değerlerinin epoch sayısına göre değişimi.



Şekil 2 incelendiğinde, kullanılan öğrenme oranına bağlı olarak hata değerlerinin epoch boyunca farklı biçimlerde değiştiği görülmektedir. Düşük öğrenme oranlarında hata değerlerinin daha yavaş azaldığı, yüksek öğrenme oranlarında ise hata değerlerinde daha belirgin değişimler olduğu gözlemlenmiştir. Elde edilen bulgular, öğrenme oranının gradyan inişi algoritmasında hata değerlerinin değişimi ile yakınsama sürecinin hızı ve kararlılığı üzerinde etkili bir parametre olduğunu göstermektedir.

Sonuç ve Tartışma:

Çalışmada, gradyan inişi algoritmasının yapay zekâ modellerinin öğrenme sürecindeki rolü matematiksel ve deneysel olarak incelenmiştir. Farklı gradyan inişi türleri ve öğrenme oranları kullanılarak gerçekleştirilen deneyler sonucunda, algoritmanın uygulanma biçiminin ve seçilen hiperparametrelerin modelin hata değerlerinin değişimi üzerinde etkili olduğu görülmüştür.

Elde edilen bulgular genel olarak değerlendirildiğinde, toplu gradyan inişi yönteminin hata değerlerinin daha kararlı bir yakınsama süreci sergilediği belirlenmiştir. Stokastik gradyan

inişi yönteminde hata değerlerinin erken aşamada minimum seviyeye ulaştığı ve eğitim süreci boyunca bu seviyede kaldığı gözlemlenmiştir. Mini-toplu gradyan inişi yöntemi ise bu iki yaklaşım arasında bir davranış göstermiştir. Bu sonuçlar, gradyan inişi türlerinin eğitim süreci üzerindeki etkilerine ilişkin literatürle uyumludur (Ruder, 2016).

Öğrenme oranı üzerine yapılan deneyler, bu parametrenin gradyan inişi algoritmasının yakınsama sürecinin hızı ve kararlılığı üzerinde belirleyici bir rol oynadığını ortaya koymuştur. Öğrenme oranının düşük olduğu durumlarda hata değerlerinin daha yavaş azaldığı, yüksek olduğu durumlarda ise hata değerlerinde daha belirgin değişimler meydana geldiği gözlemlenmiştir. Bu bulgu, öğrenme oranının uygun şekilde seçilmesinin model performansı açısından kritik olduğunu vurgulayan çalışmalarla örtüşmektedir (Goodfellow, Bengio, & Courville, 2016).

Genel olarak bu çalışma, gradyan inişi algoritmasının matematiksel yapısının ve uygulama biçiminin yapay zekâ modellerinin öğrenme süreci üzerindeki etkisini lise düzeyinde açık ve uygulamalı bir şekilde ortaya koymuştur. Çalışma, gradyan inişi türleri ve öğrenme oranı gibi temel kavramların anlaşılmasına katkı sağlamaktadır.

Gelecek çalışmalarda, farklı veri setleri, daha karmaşık modeller ve alternatif optimizasyon yöntemleri kullanılarak gradyan inişi algoritmasının davranışı daha kapsamlı biçimde incelenebilir.

Öneriler:

Bu çalışmadan elde edilen sonuçlara dayanarak, benzer bir proje yapmayı düşünen öğrenciler için bazı öneriler sunulabilir. Öncelikle, gradyan inişi algoritmasının nasıl davrandığını daha iyi anlayabilmek için farklı veri setleri kullanılarak benzer deneyler yapılabilir. Bu sayede algoritmanın farklı durumlarda nasıl sonuçlar verdiği daha net gözlemlenebilir.

Bu projede öğrenme oranı sabit tutulmuştur. Benzer çalışmalarda, farklı öğrenme oranları denenerek bu parametrenin öğrenme sürecini nasıl etkilediği daha ayrıntılı biçimde incelenebilir. Ayrıca, öğrenme oranının zamanla değiştiği yöntemler araştırılarak gradyan inişi algoritmasının daha hızlı veya daha kararlı öğrenip öğrenmediği gözlemlenebilir.

Bunun yanında, farklı hata fonksiyonları kullanılarak gradyan inişi algoritmasının değişik problem türlerinde nasıl sonuçlar verdiği incelenebilir. Özellikle sınıflandırma problemleri üzerine yapılacak çalışmalar, algoritmanın kullanım alanlarını genişletebilir. Son olarak, elde edilen sonuçların grafiklerle gösterilmesi, gradyan inişi algoritmasının öğrenme sürecinin daha kolay anlaşılmasına yardımcı olacaktır. Bu bölümde verilen önerilerin, benzer bir proje yapmayı düşünen öğrenciler için faydalı olacağı düşünülmektedir.

Kaynaklar (APA 7 Formatı):

- Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

- Ruder, S. (2016). *An overview of gradient descent optimization algorithms*. arXiv:1609.04747.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning* (2nd ed.). Springer.
- Géron, A. (2019). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.

Ekler(Varsa):

```
import numpy as np
import matplotlib.pyplot as plt

# =====
# 2. VERİ SETİ OLUŞTURMA
# =====
np.random.seed(42)

w_true = 3.0
b_true = 2.0

X = np.random.rand(100, 1) * 10
noise = np.random.randn(100, 1)
y = w_true * X + b_true + noise

X = X.flatten()
y = y.flatten()

# =====
# 3. LOSS (MSE) FONKSİYONU
# =====
def compute_loss(w, b, X, y):
    y_pred = w * X + b
    return np.mean((y_pred - y) ** 2)

# =====
# 4. BATCH GRADIENT DESCENT
# =====
def batch_gradient_descent(X, y, lr=0.001, epochs=100):
    w, b = 0.0, 0.0
    losses = []

    for epoch in range(epochs):
        y_pred = w * X + b
        dw = np.mean((y_pred - y) * X)
        db = np.mean(y_pred - y)

        w -= lr * dw
        b -= lr * db

        loss = compute_loss(w, b, X, y)
        losses.append(loss)

    return w, b, losses

# =====
# 5. STOCHASTIC GRADIENT DESCENT
# =====
def stochastic_gradient_descent(X, y, lr=0.001, epochs=100):
    w, b = 0.0, 0.0
    losses = []

    for epoch in range(epochs):
        for i in range(len(X)):
            y_pred = w * X[i] + b
            dw = (y_pred - y[i]) * X[i]
            db = y_pred - y[i]

            w -= lr * dw
            b -= lr * db

        loss = compute_loss(w, b, X, y)
        losses.append(loss)
```

```

        return w, b, losses

# =====
# 6. MINI-BATCH GRADIENT DESCENT
# =====
def mini_batch_gradient_descent(X, y, lr=0.001, epochs=100, batch_size=16):
    w, b = 0.0, 0.0
    losses = []

    for epoch in range(epochs):
        indices = np.random.permutation(len(X))
        X_shuffled = X[indices]
        y_shuffled = y[indices]

        for i in range(0, len(X), batch_size):
            X_batch = X_shuffled[i:i+batch_size]
            y_batch = y_shuffled[i:i+batch_size]

            y_pred = w * X_batch + b
            dw = np.mean((y_pred - y_batch) * X_batch)
            db = np.mean(y_pred - y_batch)

            w -= lr * dw
            b -= lr * db

            loss = compute_loss(w, b, X, y)
            losses.append(loss)

    return w, b, losses

# =====
# 7. MODELLERİ ÇALIŞTIR
# =====
w_b, b_b, loss_b = batch_gradient_descent(X, y)
w_s, b_s, loss_s = stochastic_gradient_descent(X, y)
w_m, b_m, loss_m = mini_batch_gradient_descent(X, y)

# =====
# 8. GRAFİK ÇİZİMİ
# =====
plt.figure(figsize=(8,5))
plt.plot(loss_b, label="Batch Gradient Descent")
plt.plot(loss_s, label="Stochastic Gradient Descent")
plt.plot(loss_m, label="Mini-batch Gradient Descent")
plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.title("Gradient Descent Türlerinin Karşılaştırılması")
plt.legend()
plt.show()

# =====
# 9. SON LOSS DEĞERLERİ
# =====
print("Batch GD Final Loss:", loss_b[-1])
print("Stochastic GD Final Loss:", loss_s[-1])
print("Mini-batch GD Final Loss:", loss_m[-1])

# =====
# 10. LEARNING RATE DENEYİ
# =====
learning_rates = [0.0001, 0.001, 0.01]
losses_lr = {}

for lr in learning_rates:
    _, _, losses = batch_gradient_descent(X, y, lr=lr, epochs=100)
    losses_lr[lr] = losses

# =====
# 11. LEARNING RATE KARŞILAŞTIRMA GRAFİĞİ
# =====
plt.figure(figsize=(8,5))

for lr, losses in losses_lr.items():
    plt.plot(losses, label=f"Learning Rate = {lr}")

plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.title("Learning Rate'in Gradient Descent Performansına Etkisi")
plt.legend()
plt.show()

```